

DeepPersona 코드 학습교재

모듈 04 — 임베딩 + 5:3:2 선택: `select_attributes.py` (Step 2–4)

Jin Sam Cho · cho@kaist.ac.kr

September 13, 2026

원저: Wang et al., *DeepPersona* (arXiv:2511.07338) · 논문 §3.2 "Balanced diversification"

이 모듈의 목표

한눈에 시드에 어울리는 슬롯 200개를 2,297개 중에서 고른다 — 관련성과 의외성을 5:3:2로 섞어서.

얻는 것

- 시드→요약→ `ada-002` 벡터
- `pkl` 캐시가 하는 일(속도)
- **코사인 유사도**(수식+코드)
- **5:3:2** 선택 로직 줄별 해부
- 카테고리별 비례 배분
- ▶ 이 모듈이 식 (2)의 $selector \Pr(a_i | S, P_{<i}, T)$ 에 해당.

핵심 통찰

- `near` = 상위 1/3 **결정적**, `mid`·`far` = 각 1/3에서 **랜덤**
- 페르소나당 임베딩 API **단 1번**(캐시 덕)
- `pkl` 깨지면 선택이 **전부 실패**→빈 프로필

1. 문제 — 200개를 어떻게 고르나
2. 임베딩 준비
3. 5:3:2 선택 로직
4. 카테고리별 배분
5. 실패 모드 & 재현

1. 문제 — 200개를 어떻게 고르나

관련성만? 랜덤만? 둘 다 아니다

쉽게 말하면 시드와 가까운 슬롯만 고르면 뻘한 인물(판박이). 아무거나 고르면 앞뒤가 안 맞음. 그 사이의 균형이 필요.

가까운 것만 (관련성 100%)

게임개발자 → 전부 "코딩·게임·기술" 슬롯.
깊지만 뻘함(다양성 없음).

랜덤 (관련성 0%)

게임개발자에 "양봉·발레·심해잠수" 슬롯.
다양하지만 뜬금없음(일관성 없음).

DeepPersona의 답 — 5:3:2

가까움 5 : 중간 3 : 먼 2 로 섞어, 관련성(깊이)과 의외성(다양성)을 동시에. 논문 "Balanced attribute diversification".

한눈에 핵심 함수 `_find_interesting_neighbors` — "흥미로운 이웃 찾기".

2. 임베딩 준비

시드 → 요약 텍스트 → 벡터

```
# 1) 시드 dict -> 한덩어리요약문자열
summary_parts.append(f"Age: {age}"); summary_parts.append(f"Gender: {gender}")
summary_parts.append(f"Location: {city}, {country}")
summary_parts.append(f"Career Status: {status}"); summary_parts.append(f"Values: {values}")
# ... attitude / story 등도추가 -> "\n".join(summary_parts)
# 2) 그문자열을임베딩
response = client.embeddings.create(model="text-embedding-ada-002",
                                     input=profile_summary)
embedding = np.array(response.data[0].embedding) # 차원1536 벡터
```

- **요약**: 시드의 나이·성별·위치·직업·가치관·태도·이야기를 "키: 값" 줄로 이어붙임.
- **임베딩**: 그 요약문을 text-embedding-ada-002 에 넣어 1536차원 벡터 1개를 얻음.
- **의미**: 이 벡터가 "이 사람은 의미공간의 여기쯤"이라는 좌표. 슬롯들과의 거리를 잴 기준점.
- **비용**: 페르소나당 이 호출 딱 1번(시드는 사람마다 다르므로 캐시 불가).

▶ 한국어 시드라면 여기서 영어 번역을 한 번 끼우면 정확도↑(study.md §12-3).

pkI 캐시 — 슬롯 벡터를 미리 계산해 둔다

```
with open(EMBEDDINGS_PATH, 'rb') as f:
    embeddings_data = pickle.load(f) # {'paths':[...], 'embeddings': ndarray}
    self.paths = embeddings_data['paths'] # 개 2297 슬롯경로문자열
    self.embeddings = embeddings_data['embeddings'] # (2297, 1536) float 배열
    for i, path in enumerate(self.paths):
        self.path_to_embedding[path] = self.embeddings[i] # 경로 -> 벡터 dict
```

- 구조: `paths` (2,297개 슬롯 문자열) + `embeddings` (2,297×1,536 ndarray)가 짝.
- `path_to_embedding` : "슬롯 경로 → 그 벡터" 사전. 이후 `dict lookup`으로 즉시 조회.
- 왜 캐시: 슬롯 2,297개 텍스트는 모든 페르소나가 공유·불변 → 한 번 임베딩해 저장.

페르소나 1개당	캐시 없음	캐시(pkI)
슬롯 임베딩 호출	2,297번	0번(lookup)
시드 임베딩 호출	1번	1번
10만 개 생성 비용	\$23,000	\$10

▶ 이 pkI이 깨지면(원본 repo 결함) 선택이 전부 실패 → 빈 프로필(\$5, study.md §6).

코사인 유사도 — 두 벡터의 방향 닮음

쉽게 말하면 두 화살표(벡터)가 같은 방향을 보면 1, 직각이면 0, 반대면 -1. 시드와 슬롯이 "의미상 얼마나 같은 쪽"인지를 잴다.

$$\cos(\theta) = \frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\| \|\mathbf{v}_2\|} = \frac{\sum_i v_{1i} v_{2i}}{\sqrt{\sum_i v_{1i}^2} \sqrt{\sum_i v_{2i}^2}}$$

- $\mathbf{v}_1$: 시드 벡터, $\mathbf{v}_2$: 슬롯 벡터(각 1536차원).
- 분자 $\mathbf{v}_1 \cdot \mathbf{v}_2$: 내적(같은 성분끼리 곱해 합).
- 분모 $\|\cdot\|$: 벡터 길이(정규화) → 크기 아닌 방향만 비교.
- 값 범위 $[-1, 1]$, 클수록 **의미상 가까움**.

```
norm1 = np.linalg.norm(vec1)
norm2 = np.linalg.norm(vec2)
if norm1==0 or norm2==0: return 0.0
return np.dot(vec1, vec2)/(norm1*norm2)
```

→ 코드가 위 수식 그대로. 0벡터는 0으로 방어.

3. 5:3:2 선택 로직

```
# 각슬롯의시드대비유사도계산      -> 내림차순정렬
path_similarities.sort(key=lambda x: x[1], reverse=True)
total_paths = len(path_similarities)
total_ratio = 5 + 3 + 2                                # = 10
# near: 상위에서 1/3 5/10 결정적( 상위)
near_count = min(int(target_count*5/total_ratio), total_paths//3)
near_indices = list(range(near_count))                  # 0 .. near_count
# mid: 가운데에서 1/3 3/10 랜덤()
mid_start, mid_end = total_paths//3, 2*total_paths//3
mid_count = min(int(target_count*3/total_ratio), mid_end-mid_start)
# far: 하위에서 1/3 2/10 랜덤()
far_start = 2*total_paths//3
far_count = min(int(target_count*2/total_ratio), total_paths-far_start)
```

- **정렬**: 슬롯을 시드 유사도 내림차순으로 줄 세움(0번=가장 가까움).
- **3등분**: 인덱스를 `[0,N/3)` (near) · `[N/3,2N/3)` (mid) · `[2N/3,N)` (far)로 나눔.
- **개수**: 목표 200이면 near 100 · mid 60 · far 40(=5:3:2). 단 구간 크기 초과 못 함(`min`).

▶ 이어서 각 구간에서 실제로 뽑는 방식(near=상위 그대로, mid/far=랜덤 표집)이 다음 장.

각 구간에서 어떻게 뽑나 — near는 결정적, mid·far는 랜덤

```
# near: 상위개를 near_count 순서만(섞어) 그대로
for i in random.sample(near_indices, min(near_count, len(near_indices))):
    selected_paths.append(path_similarities[i][0])
# mid / far: 각구간인덱스에서 random.sample 로무작위추출
for i in random.sample(mid_indices, min(mid_count, len(mid_indices))): ...
for i in random.sample(far_indices, min(far_count, len(far_indices))): ...
# 아직 target 미달이면 남은인덱스에서 랜덤보충
remaining = target_count - len(selected_paths)
```

- **near**: 상위 `near_count` 개를 전부(관련성은 놓치면 안 되니 결정적).
- **mid·far**: 해당 1/3 구간 안에서 무작위로 뽑음 → 사람마다 달라져 다양성↑.
- **보충**: 셋을 합쳐도 200 미달이면 남은 슬롯에서 랜덤 채움.



한눈에 같은 직업이어도 mid·far가 달라 서로 다른 200 슬롯이 된다(비교가능성 trade-off).

4. 카테고리별 배분

200개를 카테고리에 비례 배분

```
for category, paths in category_paths.items():
    category_ratio = len(paths) / len(all_paths)      # 이카테고리비중
    category_target = max(3, int(target_count*category_ratio)) # 최소개 3 보장
    category_selected = self._find_interesting_neighbors( # 카테고리별 5:3:2
        profile_embedding, paths, category_target)
    final_paths.extend(category_selected)
if len(final_paths) > target_count:                  # 초과분랜덤삭감
    final_paths = random.sample(final_paths, target_count)
```

- **비례**: 큰 카테고리(슬롯 많은)일수록 더 많이 배정. `category_ratio` = 그 카테고리 슬롯 / 전체.
- **최소 보장**: `max(3, ...)` 로 작은 카테고리도 최소 3개는 나오게.
- **5:3:2는 카테고리마다**: 각 카테고리 안에서 `_find_interesting_neighbors` 를 따로 호출.
- **삭감**: 합이 200 초과면 무작위로 줄여 정확히 맞춤.

▶ 추천 카테고리는 `process_profile()` 가 정함(`_check_if_career_needed` 로 Career 제외 가능).

5. 실패 모드 & 재현

pkl이 깨지면 — 조용히 빈 프로필

쉽게 말하면 임베딩 캐시가 없거나 손상되면 선택 함수가 빈 리스트를 반환하고, 페르소나엔 속성이 하나도 안 남는다.

연쇄

- `_load_embeddings()` 실패 → `None`
- `_find_interesting_neighbors` : "벡터DB 없음" → `return []`
- `get_top_attributes` : `return []`
- 결과 프로필 1.9KB(정상 ~27KB), 속성 0개

로그 신호

`pickle data was truncated`

有可用的的向量 据 ...返回空列表

⇒ 재현하려면 `attribute_embeddings.pkl` 이 반드시
정상이어야 함.

한눈에 재현 체크: pkl 로드 성공 확인 → 실패 시 `scripts/regenerate_embeddings.py` 로 재생성(모듈 06).

▶ 원본 repo의 pkl은 6.8MB로 잘려 있었음(정상 27MB). study.md §6 forensic 분석 참조.

꼭 기억할 것

- 시드→요약→ ada-002 벡터 1개
- 슬롯 벡터는 pk1 에 캐시(lookup)
- 유사도=코사인(방향 닮음)
- 5:3:2: near 결정적, mid·far 랜덤
- 카테고리별 비례 배분 후 200 맞춤

함수 지도

- generate_user_profile (시드, 모듈 03)
- get_selected_attributes (진입점)
- _extract_profile_summary /
_create_profile_embedding
- _compute_cosine_similarity
- _find_interesting_neighbors (5:3:2)
- get_top_attributes (배분)

한눈에 다음 05_generate_profile : 고른 슬롯에 LLM이 값을 채우고 Summary를 쓴다(Step 5).

모듈 04 끝

다음: 05_generate_profile — 깊이우선 값 채우기 (Step 5)